

~~17p~~

DSA Assignment

Aditya, Mishra
Solb
MBA TE41
DS

Q.1

Algorithm:

- 1: START
- 2: Read the array of n integers
- 3: Set $\text{largest} = -\infty$ & $\text{second_largest} = -\infty$
- 4: Traverse the array from the first element to the last
- 5: If the current element is greater than largest :
 Set $\text{second_largest} = \text{largest}$
 Set $\text{largest} = \text{current element}$
- 6: Otherwise, if the current element is smaller than largest but greater than second_largest :
 Set $\text{second_largest} = \text{current element}$
- 7: Repeat until all elements have been checked
- 8: If second_largest is still $-\infty$, report report that no second largest distinct element exists
- 9: Otherwise display second largest
- 10: STOP.

Trace: $A = [12, 35, 1, 10, 34, 1]$

Element	Largest	2 nd Largest
12	12	$-\infty$
35	35	12
1	35	12
10	35	12
34	35	34
	35	34

34

8.2 Linear Search

Algorithm:

1. START
2. Read the sorted array of product IDs & the target product ID.
3. Set $i = 0$
4. Compare $A[i]$ with the target
5. If $A[i] = \text{target}$, display that the product is found & stop.
6. Otherwise increment i by 1
7. Repeat steps 4 to 6 until the product is found or all elements are checked.
8. If the end of the array is reached display "Product not found".
9. STOP.

Trace: $ID_s = [102, 115, 123, 145, 167, 189, 201]$

Target = 167

$102 \neq 167$

$115 \neq 167$

$123 \neq 167$

$145 \neq 167$

$167 = 167 \rightarrow \text{found @ pos. 5}$

Best = $O(1)$, Avg: $O(n)$, Worst = $O(n)$

Space = $O(1)$

Binary Search Algorithm:

1. START
2. Read the sorted array
3. Set $\text{low} = 0$
4. Find $\text{mid} = \frac{\text{low} + \text{high}}{2}$
5. Compare $A[\text{mid}]$ with target
6. If $A[\text{mid}] = \text{target}$, product is found
7. If $A[\text{mid}] < \text{target}$, $\text{low} = \text{mid} + 1$
8. If $A[\text{mid}] > \text{target}$, $\text{high} = \text{mid} - 1$
9. Repeat until product is found
10. If $\text{low} > \text{high}$, product not found
11. STOP.

Trace: $ID_s = [102, 115, 123, 145, 167, 189, 201]$

Target = 167

$\text{low} = 0$

$167 > 145$

$\text{low} = 4$

$167 < 189$

$\text{low} = 4$

found

Best $OC = O(1)$

by

Binary Search Algorithm:

- 1: START
- 2: Read the sorted array & the target prod. ID
- 3: Set $low = 0$, & $high = n-1$
- 4: find $mid = (low + high) / 2$
- 5: Compare $A[mid]$ with the target
- 6: If $A[mid] = \text{target}$, display that the product is found & stop.
- 7: If $\text{target} < A[mid]$, set $high = mid - 1$
- 8: If $\text{target} > A[mid]$, set $low = mid + 1$
- 9: Repeat until $low > high$ or the product is found.
- 10: If $low > high$, display "product not found"
- 11: STOP.

Ex: IDs = [102, 115, 123, 145, 167, 189, 201]

Target = 167

$low = 0$, $high = 6$, $mid = 3$, $A[3] = 145$
 $167 > 145$, so search right half.

$low = 4$, $high = 6$, $mid = 5$, $A[5] = 189$
 $167 < 189$, so search left half.

$low = 4$, $high = 4$, $mid = 4$, $A[4] = 167$
 found @ pos. 5.

Best $O(1)$, Avg $= O(\log n)$, worst $= O(\log n)$, space $= O(1)$.

Binary search is better for a sorted database.

Q.3 Circular Queue
Assume $Front = -1$, $Rear = -1$ & Capacity = N
Is Empty Algorithm

1. START
2. Check whether $Front = -1$
3. If yes, return True
4. Otherwise, return False
5. STOP

Is Full

1. START
2. Compute $(Rear + 1) \text{ mod } N$
3. If $(Rear + 1) \text{ mod } N = Front$, return True
4. Otherwise, return False
5. STOP

EN Queue

1. START
2. Check if the queue is full
3. If full, report Queue overflow & stop
4. If the queue is empty, set $Front = 0$ & $Rear = 0$
5. Otherwise, set $Rear = (Rear + 1) \text{ mod } N$
6. Insert the element @ Queue $[Rear]$
7. STOP

Q.4 Singly linked list
Each node contains student Name & next pointer

Insert (START pos)

1. START
2. Create a node
3. Store id & name
4. Set Next next
5. Set HEAD
6. STOP

Insert (at end)

1. START
2. Create a node
3. Store id, name & Next = Null
4. If head is null & no more nodes, set head = this node
5. Otherwise, traverse the list till the last node
6. Set next of last node to this node
7. Set
8. STOP

Capacity = N

Q.4

Singly linked list

Each node contains student ID, student Name, & next pointer.

Insert: (START pos)

- 1: START
- 2: Create a node New
- 3: Store id & name in New
- 4: Set New.next = Head
- 5: Set HEAD = New
- 6: STOP

return True

Insert (at End)

- 1: START
- 2: Create a new node New
- 3: Store id, & name in New set New.next = NULL
- 4: If head = NULL, set head = New & stop
- 5: otherwise set TEMP = Head
- 6: move temp until temp.next = NULL
- 7: set TEMP.next = New
- 8: STOP.